

0.65.4. Une lettre qui avance — en simple

Le 0.65 en une ligne : `defile(0, 0, 0, ALPHABET[0], 1, 250)`. La lettre file à droite et repart de la gauche, sans fin.

1. Ce qu'il faut comprendre

C'est le 0.65, en plus simple : le chronomètre, l'effacement, `x++` et le retour à 0 sont remplacés par une seule ligne, `defile`.

`defile(numero, x, y, tuile, sens, vitesse)`, une fonction de la console (voir le 0.76.12) : au premier appel, la lettre apparaît en (`x`, `y`) ; ensuite, un pas toutes les `vitesse` ms, à droite (`sens` 1) ou à gauche (-1). Au bord, elle repart de l'autre côté : exactement le `if (x == 20) x = 0;` du 0.65.

Plus de variables à déclarer : la console retient elle-même où en est la lettre, grâce au numéro 0.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <defile>` → [0.65.3. La fonction defile\(\) — filer sur sa ligne sans arrêter le jeu](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1139 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
defile()	237 o	← main() — une fonction fournie par le langage
main()	31 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 2	26 o sur place, dans <code>defile()</code> + <code>AdresseCase</code> (10 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>defile()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)
<code>images()</code>	× 1	2 o sur place, dans <code>defile()</code>

Tes fonctions appelées :

<code>defile()</code>	× 1	24 o sur place, dans <code>main()</code>
-----------------------	-----	--

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

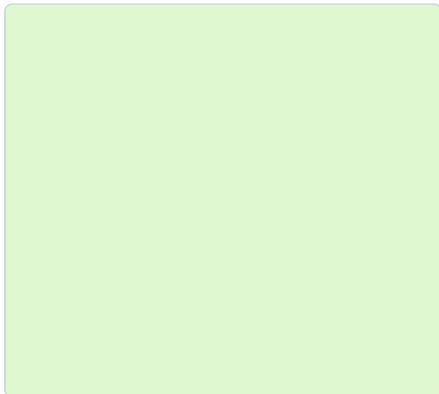
```
#include <defile>      // une tuile qui file sur sa ligne sans arrêter le jeu
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

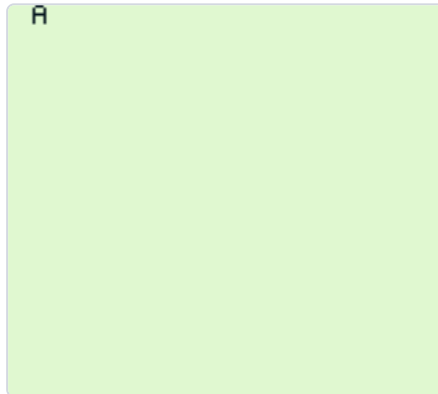
```
int main() {
    while (true) {
        image();
        // Tout le 0.65 en une ligne :
        //  numero 0, départ (0, 0), le A, vers la droite (1), 250 ms par pas
        defile(0, 0, 0, ALPHABET[0], 1, 250);
    }
}
```

3. Ce qui se passe, étape par étape

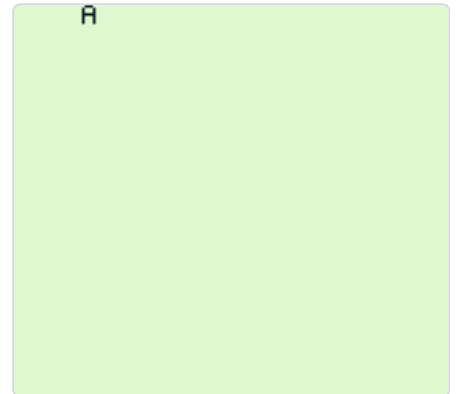
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



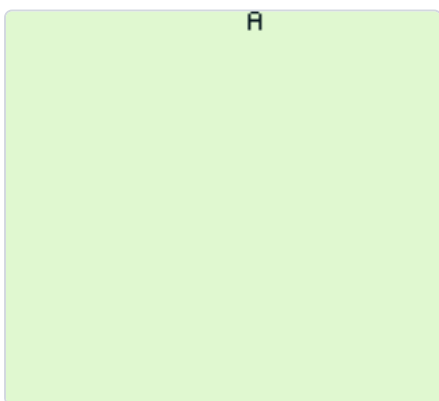
la toute première image — la console vient de démarrer — image 1



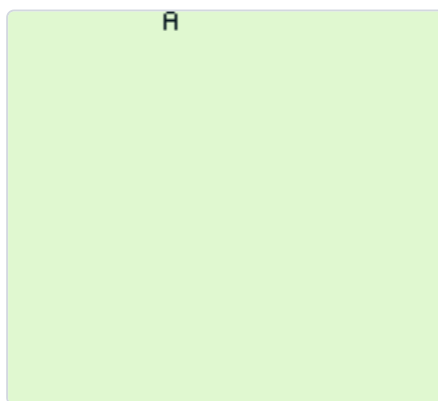
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Comme au 0.65 : un A qui avance vers la droite et repart de la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A avance vers la droite

✓ sur la ligne 0

Ensuite : **0.65.5. La fonction bouton() — lire la manette** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++